

0.27. Le carré avec deplace

Le 0.26, plus quatre lignes : avec 0 sur un axe, deplace va tout droit. Un carré à partir de (5, 5).

1. Ce qu'il faut comprendre

C'est le 0.26, plus quatre lignes : un carré, à partir de là où la diagonale s'est arrêtée.

Ce qui est nouveau ici : un 0 comme pas. `deplace(x, y, ..., 5, 0)` : 5 sur X, rien sur Y, donc tout droit vers la droite. `0, 5` : tout droit vers le bas. Avec un zéro, `deplace` fait ce que faisaient `deplace_x` ou `deplace_y`.

Le carré part de (5, 5) : à droite jusqu'en (10, 5), en bas jusqu'en (10, 10), à gauche jusqu'en (5, 10), en haut jusqu'en (5, 5).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace>` → [0.25.3. La fonction deplace\(\) — avancer sur les deux axes](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1321 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      62 o ← la console, à l'allumage – la mise en route de la console
deplace()      236 o ← main() – une fonction fournie par le langage
main()         168 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireNombre – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() – efface des cases
EcrireNombre   33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 2    26 o sur place, dans deplace() + AdresseCase (10 o)
nombre() × 2   20 o sur place, dans main() + EcrireNombre (33 o)
effacer() × 1  15 o sur place, dans deplace() + AdresseCase (10 o), EffacerCases (12 o)
image() × 2    6 o sur place, dans deplace(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace() × 5  111 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace>    // fait avancer une tuile sur les deux axes
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <nombre>      // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 0;        // la COLONNE de la lettre
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;        // la LIGNE de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    deplace(x, y, ALPHABET[0], 5, 5);    // la diagonale (le 0.26) : (5, 5)

    // Les quatre lignes ajoutées. Un 0 : cet axe ne bouge pas.
    //
    //                pasX  pasY
    deplace(x, y, ALPHABET[0], 5, 0);    // à droite : (10, 5)
    deplace(x, y, ALPHABET[0], 0, 5);    // en bas   : (10, 10)
    deplace(x, y, ALPHABET[0], -5, 0);   // à gauche : (5, 10)
    deplace(x, y, ALPHABET[0], 0, -5);   // en haut  : (5, 5)

    nombre(0, 17, x);    // 005
    nombre(4, 17, y);    // 005

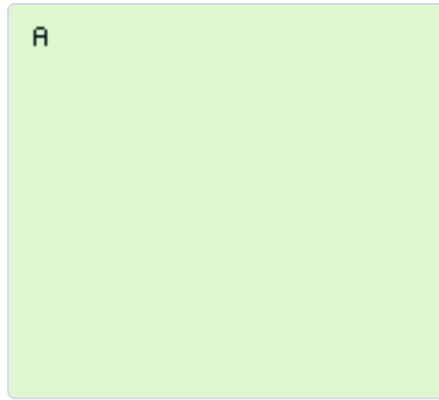
    while (true) {
        image();          // la lettre est revenue en (5, 5)
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



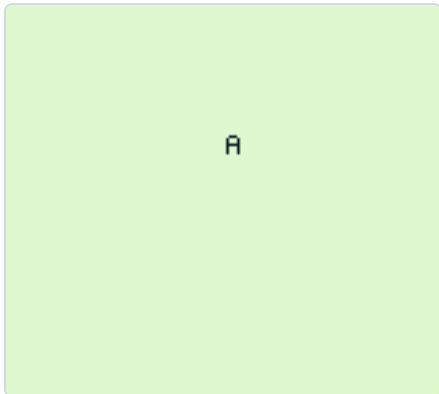
la toute première image — la console vient de démarrer — image 1



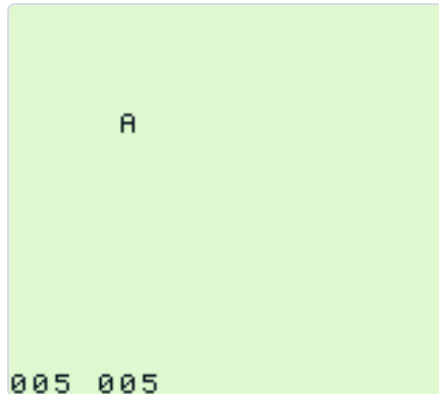
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui descend en diagonale jusqu'en (5, 5), puis fait un carré et revient en (5, 5).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les coins dans l'ordre : (10, 5), (10, 10), (5, 10) (10,5 10,10 5,10)

✓ retour en (5, 5), x et y valent 5

Ensuite : **0.27.1. Le carré avec deplace — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++